Assignment-2

Q-1) To implement Equation solver using RMI. The client should provide an equation to the Server through an interface. The server will solve the expression given by the client. (a-b)2 = a2 –2ab + b2; If a = 5 and b = 2 then return value = 52 – 2.5.2 + 22 = 9

EquationSolverServer.java

```
package rmi.equationsolver;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class EquationSolverServer extends UnicastRemoteObject implements
EquationSolverInterface {

   public EquationSolverServer() throws RemoteException {
      super();
   }

   // Formula: (a - b)^2 = a^2 - 2ab + b^2
   @Override
   public double solveEquation(double a, double b) throws RemoteException {
      double result = (a * a) - (2 * a * b) + (b * b);

      System.out.println("\n[Server] Received: a = " + a + ", b = " + b);
      System.out.println("[Server] Formula : (a-b)^2 = a^2 - 2ab + b^2");
      System.out.printf("[Server] = %.1f^2 - 2(%.1f)(%.1f) + %.1f^2%n", a, a,
b, b);
      System.out.printf("[Server] = %.1f - %.1f + %.1f = %.1f%n", a*a, 2*a*b,
b*b, result);

      return result;
   }
```

```
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            System.out.println("[Server] RMI Registry started on port 1099...");

            EquationSolverServer server = new EquationSolverServer();
            Naming.rebind("rmi://localhost:1099/EquationSolver", server);

            System.out.println("[Server] EquationSolverServer is READY.");
            System.out.println("[Server] Waiting for client...");
        } catch (Exception e) {
            System.err.println("[Server] Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

EquationSolverClient.java

```
package rmi.equationsolver;

import java.rmi.Naming;
import java.util.Scanner;

public class EquationSolverClient {

   public static void main(String[] args) {
      try {

System.out.println("==========================================")
;
         System.out.println("   RMI Equation Solver Client");
         System.out.println("   Formula: (a-b)^2 = a^2 - 2ab + b^2");

System.out.println("==========================================\n
");

         EquationSolverInterface solver =
            (EquationSolverInterface)
Naming.lookup("rmi://localhost:1099/EquationSolver");

         System.out.println("[Client] Connected to Server.\n");

         Scanner sc = new Scanner(System.in);

         System.out.print("[Client] Enter value of a: ");
         double a = sc.nextDouble();

         System.out.print("[Client] Enter value of b: ");
         double b = sc.nextDouble();

         double result = solver.solveEquation(a, b);

         System.out.println("\n----------------------------------------");
         System.out.printf("[Client] (%.1f - %.1f)^2 = %.2f%n", a, b, result);
         System.out.println("----------------------------------------");

         sc.close();
```

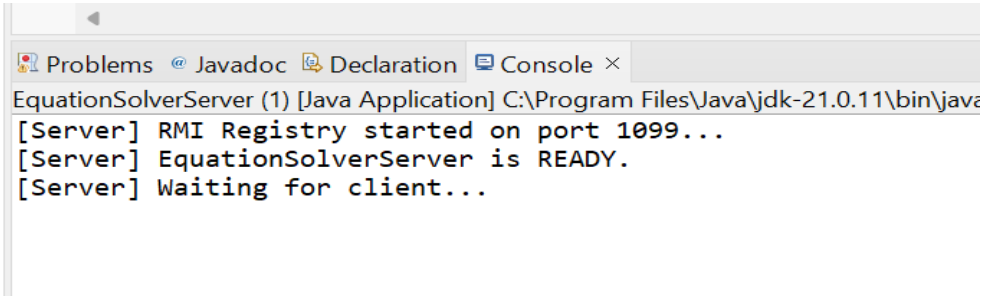
```
        } catch (Exception e) {
            System.err.println("[Client] Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

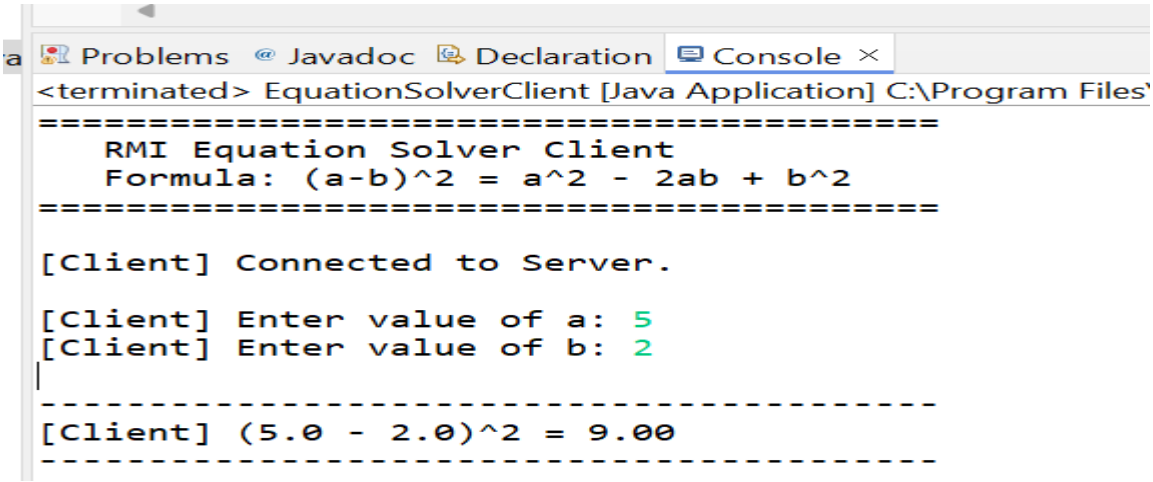
EquationSolverInterface

```
package rmi.equationsolver;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface EquationSolverInterface extends Remote {
    double solveEquation(double a, double b) throws RemoteException;
}
```

Name: Prajwal Nevase Roll No : 36 Div : B2

Output





2) Implement calculator

CalculatorServer.java

```
package rmi.calculator;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class CalculatorServer extends UnicastRemoteObject implements
CalculatorInterface {

    public CalculatorServer() throws RemoteException {
        super();
    }

    @Override
    public double add(double a, double b) throws RemoteException {
        System.out.printf("[Server] ADD: %.2f + %.2f = %.2f%n", a, b, a + b);
        return a + b;
    }

    @Override
    public double subtract(double a, double b) throws RemoteException {
        System.out.printf("[Server] SUB: %.2f - %.2f = %.2f%n", a, b, a - b);
        return a - b;
    }

    @Override
    public double multiply(double a, double b) throws RemoteException {
        System.out.printf("[Server] MUL: %.2f * %.2f = %.2f%n", a, b, a * b);
        return a * b;
    }

    @Override
    public double divide(double a, double b) throws RemoteException {
        if (b == 0) throw new RemoteException("Division by zero not allowed.");
        System.out.printf("[Server] DIV: %.2f / %.2f = %.2f%n", a, b, a / b);
        return a / b;
    }

    @Override
```

```
    public double modulus(double a, double b) throws RemoteException {
        if (b == 0) throw new RemoteException("Modulus by zero not allowed.");
        System.out.printf("[Server] MOD: %.2f %% %.2f = %.2f%n", a, b, a % b);
        return a % b;
    }

    @Override
    public double power(double base, double exp) throws RemoteException {
        double result = Math.pow(base, exp);
        System.out.printf("[Server] POW: %.2f ^ %.2f = %.2f%n", base, exp,
result);
        return result;
    }

    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            System.out.println("[Server] RMI Registry started on port 1099...");

            CalculatorServer server = new CalculatorServer();
            Naming.rebind("rmi://localhost:1099/Calculator", server);

            System.out.println("[Server] CalculatorServer is READY.");
            System.out.println("[Server] Waiting for client...");
        } catch (Exception e) {
            System.err.println("[Server] Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

CalculatorClient.java

```
package rmi.calculator;

import java.rmi.Naming;
import java.util.Scanner;

public class CalculatorClient {

    public static void main(String[] args) {
        try {
```

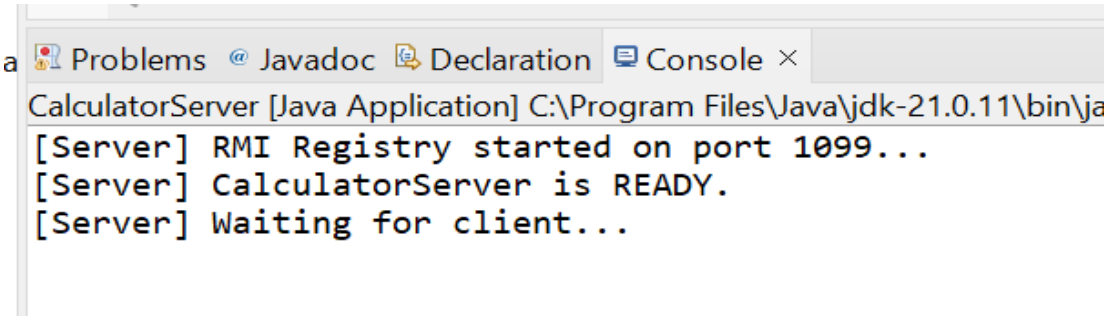
```
System.out.println("==========================================")
;
        System.out.println("       RMI Calculator Client");

System.out.println("==========================================\n
");

        CalculatorInterface calc =
           (CalculatorInterface)
Naming.lookup("rmi://localhost:1099/Calculator");

        System.out.println("[Client] Connected to Calculator Server.\n");

        Scanner sc = new Scanner(System.in);
        boolean running = true;

        while (running) {
           System.out.println("----------------------------------------");
           System.out.println("1. Addition (+)");
           System.out.println("2. Subtraction (-)");
           System.out.println("3. Multiplication (*)");
           System.out.println("4. Division (/)");
           System.out.println("5. Modulus (%)");
           System.out.println("6. Power (^)");
           System.out.println("7. Exit");
           System.out.print("Choice [1-7]: ");

           int choice = sc.nextInt();
           if (choice == 7) { running = false; break; }

           System.out.print("Enter a: ");
           double a = sc.nextDouble();
           System.out.print("Enter b: ");
           double b = sc.nextDouble();

           double result = 0;
           String op = "";

           switch (choice) {
              case 1: result = calc.add(a, b);      op = a + " + " + b; break;
              case 2: result = calc.subtract(a, b); op = a + " - " + b; break;
              case 3: result = calc.multiply(a, b); op = a + " * " + b; break;
```

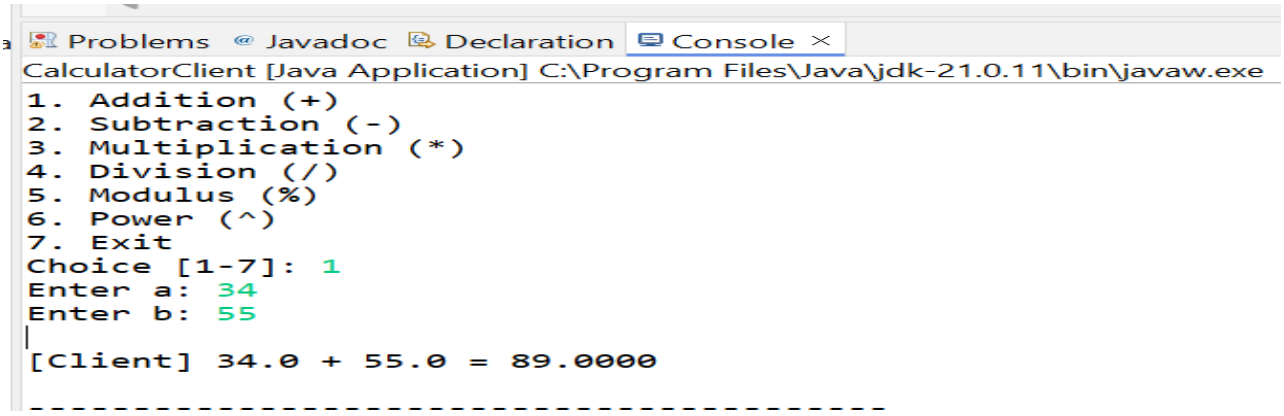
```
                case 4: result = calc.divide(a, b);   op = a + " / " + b; break;
                case 5: result = calc.modulus(a, b);  op = a + " % " + b; break;
                case 6: result = calc.power(a, b);    op = a + " ^ " + b; break;
                default: System.out.println("Invalid!"); continue;
            }
            System.out.printf("%n[Client] %s = %.4f%n%n", op, result);
        }
        sc.close();

    } catch (Exception e) {
        System.err.println("[Client] Error: " + e.getMessage());
        e.printStackTrace();
    }
  }
}
```

CalculatorInterface

```
package rmi.calculator;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface CalculatorInterface extends Remote {
   double add(double a, double b)       throws RemoteException;
   double subtract(double a, double b)  throws RemoteException;
   double multiply(double a, double b)  throws RemoteException;
   double divide(double a, double b)    throws RemoteException;
   double modulus(double a, double b)   throws RemoteException;
   double power(double base, double exp) throws RemoteException;
}
```

Output:





Name: Prajwal Nevase Roll No : 36 Div : B2

```
Choice [1-7]: 2
Enter a: 50
Enter b: 23

[Client] 50.0 - 23.0 = 27.0000

----------------------------------------
```

```
Choice [1-7]: 3
Enter a: 4
Enter b: 5

[Client] 4.0 * 5.0 = 20.0000
```

```
Choice [1-7]: 4
Enter a: 4
Enter b: 2

[Client] 4.0 / 2.0 = 2.0000

----------------------------------------
```

Problems Javadoc Declaration Console ×

CalculatorServer [Java Application] C:\Program Files\Java\jdk-21.0.11\bin\ja

```
[Server] RMI Registry started on port 1099...
[Server] CalculatorServer is READY.
[Server] Waiting for client...
[Server] ADD: 34.00 + 55.00 = 89.00
[Server] SUB: 50.00 - 23.00 = 27.00
[Server] MUL: 4.00 * 5.00 = 20.00
[Server] DIV: 4.00 / 2.00 = 2.00
[Server] MOD: 56.00 % 23.00 = 10.00
```

3) Solve multiple star pattern (rightTriangle, pyramid, diamond)
StarPatternServer.java

```
package rmi.starpattern;

import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class StarPatternServer extends UnicastRemoteObject implements
StarPatternInterface {

   public StarPatternServer() throws RemoteException {
      super();
   }

   // Right Triangle Pattern
   @Override
   public String rightTriangle(int rows) throws RemoteException {
      System.out.println("[Server] Generating Right Triangle, rows=" + rows);
      StringBuilder sb = new StringBuilder("Right Triangle:\n");
      for (int i = 1; i <= rows; i++) {
         for (int j = 1; j <= i; j++) sb.append("* ");
         sb.append("\n");
      }
      return sb.toString();
   }

   // Pyramid Pattern
   @Override
   public String pyramid(int rows) throws RemoteException {
      System.out.println("[Server] Generating Pyramid, rows=" + rows);
      StringBuilder sb = new StringBuilder("Pyramid:\n");
      for (int i = 1; i <= rows; i++) {
         for (int s = 1; s <= rows - i; s++) sb.append(" ");
         for (int j = 1; j <= (2 * i - 1); j++) sb.append("*");
         sb.append("\n");
      }
      return sb.toString();
   }

   // Diamond Pattern
```

```
    @Override
    public String diamond(int rows) throws RemoteException {
        System.out.println("[Server] Generating Diamond, rows=" + rows);
        StringBuilder sb = new StringBuilder("Diamond:\n");
        // Upper half
        for (int i = 1; i <= rows; i++) {
            for (int s = 1; s <= rows - i; s++) sb.append(" ");
            for (int j = 1; j <= (2 * i - 1); j++) sb.append("*");
            sb.append("\n");
        }
        // Lower half
        for (int i = rows - 1; i >= 1; i--) {
            for (int s = 1; s <= rows - i; s++) sb.append(" ");
            for (int j = 1; j <= (2 * i - 1); j++) sb.append("*");
            sb.append("\n");
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            System.out.println("[Server] RMI Registry started on port 1099...");

            StarPatternServer server = new StarPatternServer();
            Naming.rebind("rmi://localhost:1099/StarPattern", server);

            System.out.println("[Server] StarPatternServer is READY.");
            System.out.println("[Server] Waiting for client...");
        } catch (Exception e) {
            System.err.println("[Server] Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

StarPatternClient.java

```
package rmi.starpattern;
import java.rmi.Naming;
import java.util.Scanner;

public class StarPatternClient {

   public static void main(String[] args) {
      try {

System.out.println("==========================================")
;
         System.out.println("   RMI Star Pattern Generator Client");

System.out.println("==========================================\n
");

         StarPatternInterface pattern =
            (StarPatternInterface)
Naming.lookup("rmi://localhost:1099/StarPattern");

         System.out.println("[Client] Connected to StarPattern Server.\n");

         Scanner sc = new Scanner(System.in);
         boolean running = true;

         while (running) {
            System.out.println("----------------------------------------");
            System.out.println("1. Right Triangle");
            System.out.println("2. Pyramid");
            System.out.println("3. Diamond");
            System.out.println("4. Show All Patterns");
            System.out.println("5. Exit");
            System.out.print("Choice [1-5]: ");

            int choice = sc.nextInt();
            if (choice == 5) { running = false; break; }

            System.out.print("Enter number of rows: ");
```

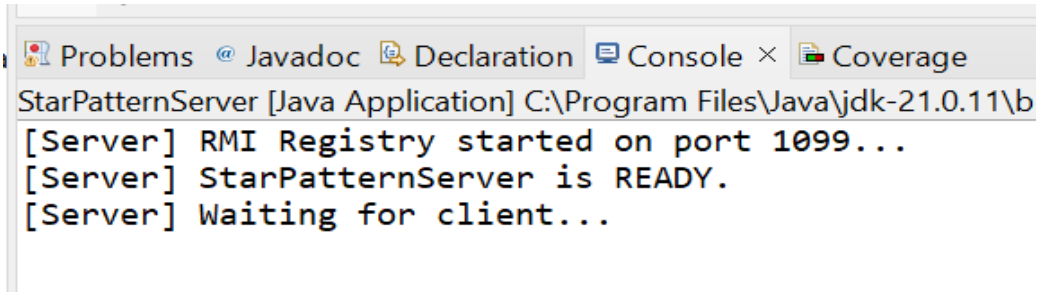
```
            int rows = sc.nextInt();


System.out.println("\n==========================================
");

            switch (choice) {
               case 1: System.out.println(pattern.rightTriangle(rows)); break;
               case 2: System.out.println(pattern.pyramid(rows));       break;
               case 3: System.out.println(pattern.diamond(rows));       break;
               case 4:
                  System.out.println(pattern.rightTriangle(rows));
                  System.out.println(pattern.pyramid(rows));
                  System.out.println(pattern.diamond(rows));
                  break;
               default: System.out.println("Invalid choice!"); continue;
            }


System.out.println("==========================================\n
");
         }
         sc.close();

      } catch (Exception e) {
         System.err.println("[Client] Error: " + e.getMessage());
         e.printStackTrace();
      }
   }
}
```

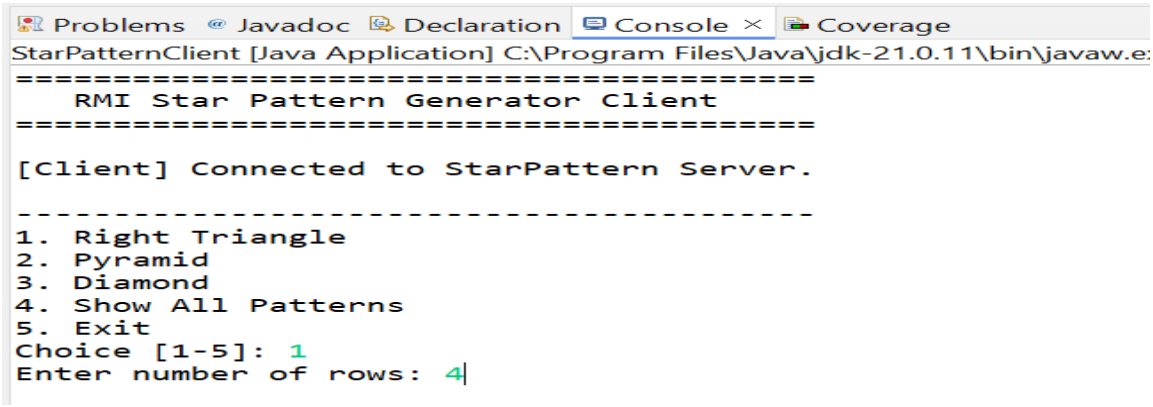
StarPatternInterface

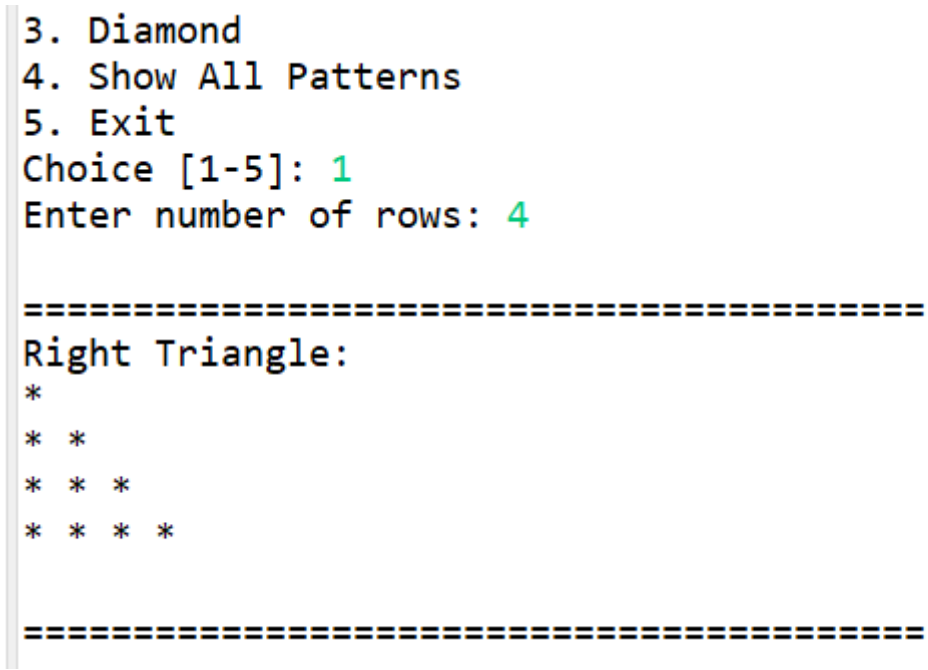
```
package rmi.starpattern;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface StarPatternInterface extends Remote {
   String rightTriangle(int rows) throws RemoteException;
   String pyramid(int rows)       throws RemoteException;
   String diamond(int rows)       throws RemoteException;
}
```

Name: Prajwal Nevase Roll No : 36 Div : B2

Output:







Name: Prajwal Nevase Roll No : 36 Div : B2

```
----------------------------------------
1. Right Triangle
2. Pyramid
3. Diamond
4. Show All Patterns
5. Exit
Choice [1-5]: 2
Enter number of rows: 3

========================================
Pyramid:
  *
 ***
*****

----------------------------------------
```

```
----------------------------------------
1. Right Triangle
2. Pyramid
3. Diamond
4. Show All Patterns
5. Exit
Choice [1-5]: 3
Enter number of rows: 5

========================================
Diamond:
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```